OAuth & Third-Party App Access Governance for Google Workspace

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

- What this solution is . . . 4
- How it works . . . 5
- Reference design . . . 6
- Benefits . . . 6
- Limitations & boundaries . . . 7
- Deployment model . . . 7

Requirements 8

- 1. Super-admin (or delegated admin with API-controls privilege) . . . 8
- 2. Admin SDK API enabled + a delegated identity for automation . . . 8
- 3. Scope-sensitivity reference (Google's OAuth scope classification) . . . 8
- 4. A defined app-vetting decision authority and risk criteria . . . 9
- 5. A pilot OU/group representative of real app usage . . . 9
- 6. Change-control window for the org-wide access-control change . . . 9

Global Requirements — Google Workspace & Gemini Solutions 10

- 1. Google Workspace / Cloud Identity tenant . . . 10
- 2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 10
- 3. DNS control for the routed domains . . . 11
- 4. Identity & provisioning source of truth . . . 11
- 5. A pilot scope (OU / group) and change control . . . 11
- 6. Security, compliance & governance stakeholders . . . 12

Implementation 12

- Stage 0 — Prepare the automation environment . . . 12
- Stage 1 — Assess: inventory connected apps + scopes . . . 12
- Stage 2 — Design: classify apps and define policy . . . 13
- Stage 3 — Configure (preview first): API access control, Marketplace, unverified apps . . . 13
- Stage 4 — Pilot: apply to the pilot OU/group . . . 14
- Stage 5 — Validate against the gates . . . 14
- Stage 6 — Expand org-wide under change control . . . 15

Testing & Validation 15

- Test 1 — Inventory completeness . . . 15
- Test 2 — Gate A: high-risk scopes only held by Trusted apps . . . 15
- Test 3 — Gate B: unverified/unconfigured apps blocked . . . 16
- Test 4 — Gate C: Marketplace allowlist enforced . . . 16

Test 5 — Trusted app still works (no false breakage) . . . 16
Test 6 — Risky-grant revocation (dry-run vs apply) . . . 16
Test 7 — Vetting workflow enforced for a new app . . . 17
Troubleshooting 17
Auth & API access . . . 17
Inventory quality . . . 18
Configuration & policy . . . 18
Revocation . . . 19
Best Practices 19
Governance & process . . . 19
Ongoing hygiene . . . 20
Safety & operations . . . 20
Alignment . . . 20
Deliverables Summary 21
Documentation . . . 21
Python tools (`scripts/`) . . . 21
Appendix A — PowerShell Scripts 23

Executive Summary

Outcome. This solution brings the organization's largest invisible attack surface — the OAuth grants that third-party applications hold against Google Workspace data — under explicit, auditable governance. On completion, every connected app and OAuth scope is inventoried and risk-scored, only vetted apps are trusted, unverified apps and high-risk scopes are blocked by default, risky standing tokens are revoked with evidence, and a repeatable vetting workflow governs every new app going forward.

The problem. In a large Google Workspace estate, thousands of users grant hundreds of external apps standing access to mail, Drive, calendars, and directory data with a single "Allow" click. These tokens survive password resets, are invisible to most endpoint controls, and are rarely reviewed. One over-scoped or compromised vendor can exfiltrate data at scale. Most organizations cannot even name which apps hold restricted scopes today.

The approach. Using Google's **native API access control** model and the **Admin SDK** (Reports and Directory APIs) driven by **Python** automation, the solution: (1) inventories connected apps and scopes and risk-scores them; (2) classifies apps **Trusted / Limited / Blocked**; (3) configures **Marketplace allowlisting** and **unverified-app blocking**; (4) reviews and revokes high-risk tokens; and (5) stands up an **app-vetting/approval workflow**. It follows a strict **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model with explicit pass/fail gates and a documented rollback for every org-wide change.

Why it is safe and governed. Nothing is changed org-wide before it is proven on a representative pilot OU. The included Python tools are **read-only or dry-run by default**; every write and every token revocation requires an explicit `--apply` flag and approver sign-off, and produces an evidence log. No credentials or service-account keys live in code — automation uses Application Default Credentials scoped to the minimum read-only APIs. Legal and compliance determinations remain with the organization's security, legal, and data-governance functions; this solution provides the technical inventory, controls, and evidence they need.

Deliverables. A complete implementation guide, requirements and testing procedures, best-practice guidance, and **five Python tools** (inventory, access-control config helper, validation/report generator, risky-grant revoker, and a scope-config helper) — everything a competent cloud/identity engineer needs to run the governance to completion. Self-guided and tested, at a fraction of the cost of a bespoke consulting engagement.

Solution Overview

What this solution is

Every time a user clicks **"Allow"** on a third-party application's consent screen, they grant that app an OAuth token that can read, write, or delete Google Workspace data on their behalf — mail, Drive files, calendars, contacts, and more. Across an enterprise of 80,000–150,000+ users, this produces thousands of standing grants to hundreds of external applications, most of them never reviewed. It is one of the largest and least visible attack surfaces in a Google Workspace estate: a single over-scoped or malicious app, or a compromised vendor, can exfiltrate data at scale using tokens that survive password changes and are invisible to most endpoint controls.

This solution establishes **OAuth & third-party app-access governance** for Google Workspace. It brings the invisible consent surface under explicit control by:

- **Inventorying connected third-party apps and their OAuth scopes** — who has granted what to which

app, and how sensitive each scope is.

- **Configuring API access control** — the Google Admin console's **trusted / limited / blocked**

model for OAuth apps, **Google Workspace Marketplace allowlisting**, and **blocking of unverified (unconfigured) apps** and their sensitive-scope access.

- **Reviewing high-risk scopes and tokens** — surfacing the grants that carry restricted or

sensitive scopes (full Gmail, Drive, admin, directory) and the tokens that should be revoked.

- **Standing up an app-vetting / approval workflow** — a repeatable process (risk criteria,

reviewers, decision record) so new third-party apps are assessed *before* they are trusted, not discovered after an incident.

The result: only trusted, vetted applications reach the organization's Google data, and every standing grant is inventoried, risk-scored, and governed.

How it works

The solution operates against the **Google Admin console** (admin.google.com) and the **Admin SDK** (the **Reports API** for OAuth-token activity and the **Directory API** for token/domain-app inventory), driven by **Python** automation (the `google-api-python-client` client library and, where useful, `gcloud` subprocess wrappers) authenticated with **Application Default Credentials (ADC)** or a delegated service account — never hardcoded keys.

Control is exercised through **Google's native API access control model**, configured under **Admin console** → **Security** → **Access and data control** → **API controls**:

- **App access control** classifies each OAuth/OIDC app as **Trusted**, **Limited**, or **Blocked**.

"Trusted" apps may access **all** scopes (including restricted/sensitive ones); "Limited" apps may access only non-sensitive/basic scopes; "Blocked" apps get no access.

- **Unconfigured (unverified) third-party apps** can be governed globally: you can **block** their

access to Google services entirely, or restrict them from **sensitive and restricted scopes**, so a newly encountered app cannot silently obtain high-risk access.

- **Google Workspace Marketplace settings** control whether users can install Marketplace apps

freely, only from an **allowlist**, or not at all.

- **Domain-wide OAuth token inventory** (Directory API `tokens` resource) and **OAuth-token log

events** (Reports API) provide the connected-app + scope inventory and the revocation surface.

The four capabilities are layered: **inventory** feeds **risk classification**, which drives **access control configuration** and **token revocation**, and the **vetting workflow** governs everything new going forward. Every change is applied **pilot-first** with explicit pass/fail gates and a documented rollback.

Reference design

The governance model is organized around named artifacts (reused verbatim throughout implementation, testing, and the scripts):

Artifact	Name convention	Purpose
App inventory	`App Inventory – Org-Wide – v1`	Connected apps + scopes + grant counts, risk-scored
Trusted-app list	`Trusted Apps – Org-Wide – v1`	Vetted apps allowed all/restricted scopes
Limited-app list	`Limited Apps – Org-Wide – v1`	Apps allowed basic/non-sensitive scopes only
Blocked-app list	`Blocked Apps – Org-Wide – v1`	Apps denied all Google access
Unverified-app policy	`Unverified-App Block – Org-Wide – Pilot`	Global block/limit of unconfigured apps' sensitive scopes
Marketplace policy	`Marketplace Allowlist – Org-Wide – v1`	Allowlist-only Marketplace install setting
Pilot scope	`App-Access Pilot – Pilot-OU – v1`	OU/group the controls are validated on first
Vetting workflow	`App-Vetting Workflow – Org-Wide – v1`	Risk criteria + reviewers + decision record for new apps
Revocation runbook	`Token Revocation – Risky-Grants – Runbook`	Steps + evidence to revoke high-risk grants

The **risk score** for a connected app is derived from the sensitivity of the scopes it holds (restricted > sensitive > basic), the number of users who granted it, whether it is verified by Google, and whether it appears on the Trusted list. Scope-sensitivity tiers follow Google's own classification (restricted and sensitive scopes are enumerated in Google's OAuth API scopes documentation — *verify the current list for your tenant*, as Google updates it).

Benefits

• **Visibility:** a complete, risk-scored inventory of every third-party app and OAuth scope touching Google data — replacing an unknown surface with a governed one.

• **Containment:** unverified apps and high-risk scopes are blocked by default; only vetted apps are trusted.

• **Least privilege for apps:** the trusted/limited/blocked model enforces that apps get only the scope tier they were vetted for.

• **Repeatability:** a documented vetting workflow means new-app risk is assessed before access is

granted, not after.

• **Auditability:** every configuration and revocation is named, evidenced, and reproducible via the included Python tools.

Limitations & boundaries

• **Native Google controls only.** This solution uses Google's built-in API access control, Marketplace, and Admin SDK capabilities. A dedicated SaaS-security / SSPM product (or Google's own advanced app-governance tooling where licensed) may add deeper behavioral detection; that is complementary, not replaced here. *Verify which capabilities your Workspace edition includes.*

• **Edition-dependent features.** Some app-access and security-center capabilities depend on the Workspace edition (e.g. Enterprise tiers). Where a control's availability is edition- or rollout-dependent, this guide flags "verify in your tenant" rather than asserting it.

• **Scope classification evolves.** Google's list of sensitive/restricted scopes changes; the risk model reads the current classification and flags unknown scopes for manual review rather than assuming a fixed list.

• **Legal/compliance determinations deferred.** Whether a given app or data flow is *permitted* under the organization's policy or regulatory regime is a decision for security/legal/data-governance stakeholders; this solution provides the technical inventory, controls, and evidence.

• **User-consent behavior.** Blocking apps changes what users can install; expansion beyond the pilot must be communicated (see change management) to avoid support load.

Deployment model

This solution follows the house model: **Assess** → **(Design** →**) Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback for every org-wide change:

1. **Assess** — inventory connected apps + scopes with the read-only discovery tool; risk-score them.

2. **Design** — classify apps into Trusted / Limited / Blocked; define unverified-app and Marketplace policy; define the vetting workflow and risk criteria.

1. **Configure** — apply API access control, Marketplace allowlist, and unverified-app block in the Admin console (dry-run/preview first).

1. **Pilot** — apply to `App-Access Pilot – Pilot-OU – v1` first; validate no legitimate app is broken.

1. **Validate** — run the validation tool against the pass/fail gates (high-risk scopes controlled, unverified apps blocked, approvals enforced).

1. **Expand** — roll out org-wide under change control; operationalize the vetting workflow and periodic token review.

Requirements

These are the **solution-specific** prerequisites for OAuth & third-party app-access governance. They build on the shared **Global Requirements — Google Workspace & Gemini Solutions** (`global-requirements-google.md`), which already cover the Workspace/Cloud Identity tenant with a super-admin, the identity source of truth, a pilot OU/group and change control, and security/compliance stakeholders. Do **not** re-establish those here; verify them, then confirm the additional items below.

Each requirement lists **Description / Why required / How to verify / How to implement if missing.**

1. Super-admin (or delegated admin with API-controls privilege)

• **Description:** A Google Workspace **super-admin** account, or a custom admin role that includes the **Services** → **Security** and the **API controls / App access control** management privileges.

• **Why required:** Configuring app access control (trusted/limited/blocked), Marketplace settings, and the unverified-app block are performed in **Admin console** → **Security** → **Access and data control** → **API controls**, which requires these privileges.

• **How to verify:** Sign in to admin.google.com; confirm you can open **Security → Access and data control → API controls **and see** App access control **and** Settings**.

• **How to implement if missing:** Have a super-admin assign the role (**Account** → **Admin roles**), or perform the work as super-admin under change control.

2. Admin SDK API enabled + a delegated identity for automation

• **Description:** The **Admin SDK API** enabled for the tenant, and an identity the Python tools can authenticate as — either your admin user via **Application Default Credentials** (`gcloud auth application-default login`) with the required OAuth scopes, or a **service account with domain-wide delegation** authorized for the read-only reporting/directory scopes.

• **Why required:** The inventory, validation, and report tools call the **Reports API** (`admin.reports.audit.readonly`) for OAuth-token activity and the **Directory API** (`admin.directory.user.security` for the `tokens` resource) to enumerate and revoke grants.

• **How to verify:** In the Google Cloud Console for the automation project, confirm **Admin SDK API** is enabled; run one of the included discovery tools with `--check-auth` — it prints the authenticated principal and the granted scopes.

• **How to implement if missing:** Enable the Admin SDK API in the automation project; for a service account, authorize domain-wide delegation in **Admin console** → **Security** → **Access and data control** → **API controls** → **Domain-wide delegation** for the specific read-only scopes. **Never export or commit a service-account key** — prefer ADC or workload identity.

3. Scope-sensitivity reference (Google's OAuth scope classification)

• **Description:** The current mapping of OAuth scopes to Google's **restricted / sensitive / basic** sensitivity tiers, used to risk-score connected apps.

• **Why required:** The risk model classifies each app by the most sensitive scope it holds; this depends on Google's published scope classification, which Google updates over time.

• **How to verify:** Cross-check a sample of observed scopes against Google's **OAuth API scopes** and **restricted/sensitive scopes** documentation. *Verify the current classification for your tenant — do not assume a frozen list.*

• **How to implement if missing:** Use the included scope-config helper to load a reviewable scope-tier reference file; any scope not in the reference is flagged **UNKNOWN** for manual triage rather than silently treated as low-risk.

4. A defined app-vetting decision authority and risk criteria

• **Description:** A named **approver/reviewer** (security or IT-governance) and agreed **risk criteria** for deciding whether a third-party app becomes Trusted, Limited, or Blocked (data sensitivity, vendor security posture, scope necessity, verification status, business owner).

• **Why required:** The vetting workflow (`App-Vetting Workflow – Org-Wide – v1`) requires a decision authority and objective criteria; otherwise "trusted" is arbitrary.

• **How to verify:** Confirm the approver(s) and a written risk-criteria checklist exist and are agreed with the security stakeholder from the shared requirements.

• **How to implement if missing:** Define the criteria and approver before configuring the Trusted list; the included report generator produces the review packet the approver signs off.

5. A pilot OU/group representative of real app usage

• **Description:** The pilot scope (`App-Access Pilot – Pilot-OU – v1`) — an OU or group whose members actually use a representative set of third-party apps (not a dormant test OU).

• **Why required:** Blocking unverified apps or tightening Marketplace can break legitimate workflows; the pilot must exercise real apps so breakage is caught before org-wide expansion.

• **How to verify:** Confirm the pilot OU/group contains active users whose connected-app inventory (from the discovery tool) covers the common business apps.

• **How to implement if missing:** Select or assemble a representative pilot cohort with the business and change-management owners; extend the shared pilot OU/group if it is not representative.

6. Change-control window for the org-wide access-control change

• **Description:** An approved **change window and rollback plan** for switching the org-wide unverified-app policy and Marketplace setting from their current (often permissive) state to the governed state.

• **Why required:** These are tenant-wide settings that change what every user can install/authorize; they must be scheduled, communicated, and reversible.

• **How to verify:** Confirm a change ticket exists referencing `Unverified-App Block – Org-Wide – Pilot` → org-wide expansion, with the documented rollback (revert the setting) attached.

• **How to implement if missing:** Raise the change with the change-control process from the shared requirements; do not flip org-wide settings ad hoc.

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model;
the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the
owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

Implementation

Follow these steps in order. They implement the **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model, pilot-first, with explicit pass/fail gates and a documented rollback for every org-wide change. Every artifact created is **named** with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in later steps, testing, troubleshooting, and the Python tools.

Console navigation is current as of authoring; **verify paths in your tenant** where Google may have moved a menu. All automation is the included Python tooling, run with ADC or a delegated service account — **no keys in code**.

Stage 0 — Prepare the automation environment

1. On the workstation that will run the tools, install Python 3.10+ and the client library:

```
pip install google-api-python-client google-auth
```

1. Authenticate with Application Default Credentials as an admin (or configure the delegated service account per Requirement 2):

```
gcloud auth application-default login
```

1. Confirm the automation identity and scopes:

```
python oauth-app-inventory.py --check-auth
```

Pass gate: the tool prints the authenticated principal and confirms the read-only Reports and Directory scopes are present. If not, fix auth before proceeding (see Requirement 2).

Stage 1 — Assess: inventory connected apps + scopes

Produce `App Inventory – Org-Wide – v1`.

1. Run the read-only discovery tool to enumerate connected third-party apps, their OAuth scopes, and

the number of users who granted each — risk-scored by scope sensitivity:

```
python oauth-app-inventory.py --days 180 --out app-inventory.json
```

It reads OAuth-token activity from the **Reports API** and domain tokens from the **Directory API**, joins them, and applies the scope-sensitivity reference to score each app (**HIGH / MEDIUM / LOW / UNKNOWN**).

1. Review `app-inventory.json`. Confirm every **HIGH**-risk app (holding restricted/sensitive scopes:

full Gmail, full Drive, admin/directory, etc.) is understood, and every **UNKNOWN**-scope app is triaged against the scope reference.

Pass gate: the inventory is complete (row count matches the Admin console **API controls** → **App access control** → **connected apps** count, allowing for reporting lag) and no HIGH/UNKNOWN app is unexplained.

Stage 2 — Design: classify apps and define policy

Produce `Trusted Apps – Org-Wide – v1`, `Limited Apps – Org-Wide – v1`, `Blocked Apps – Org-Wide – v1`, and the vetting workflow.

1. Using the inventory + Requirement 4 risk criteria, classify each app into:

• **Trusted** — vetted, may access all (incl. restricted/sensitive) scopes → `Trusted Apps –

Org-Wide – v1`.

• **Limited** — allowed basic/non-sensitive scopes only → `Limited Apps – Org-Wide – v1`.

• **Blocked** — denied all Google access → `Blocked Apps – Org-Wide – v1`.

1. Record each decision (app name, OAuth client ID, scopes, risk, decision, approver, date) — the

report generator produces this review packet for the approver.

1. Define `App-Vetting Workflow – Org-Wide – v1`: the intake form, risk criteria, reviewer, decision

record, and where the decision is applied (which list). This is the durable process for all new apps going forward.

Pass gate: every HIGH-risk app has an explicit Trusted/Limited/Blocked decision with a named approver; the vetting workflow is documented and agreed.

Stage 3 — Configure (preview first): API access control, Marketplace, unverified apps

All configuration lives in **Admin console** → **Security** → **Access and data control** → **API controls**. Configure with a dry-run/preview mindset — stage the desired state, apply to the pilot first.

1. **App access control (per-app trusted/limited/blocked).** In **API controls → App access

control → Manage third-party app access**, set each app from your Stage 2 lists to** Trusted**,** Limited**, or** Blocked**. You can scope some settings to an OU/group for the pilot.

• Preview the intended state first with the config helper (dry-run default):

```
python api-access-control-helper.py --plan classification.json --dry-run
```

It prints, per app, the current vs. intended access level and flags any change that would break a legitimate app — **no change is applied without** `--apply`**.**

1. **Marketplace allowlist.** In **API controls → App access control → Settings → Google Workspace

Marketplace settings**, set the install policy to** "Allow users to install only allowlisted apps"** and populate the allowlist from `Trusted Apps – Org-Wide – v1`. Name this `Marketplace Allowlist – Org-Wide – v1`.

1. **Unverified / unconfigured third-party apps.** In **API controls → Settings → Unconfigured third-

party apps **(naming varies by tenant/rollout — *verify in your tenant*), set unconfigured apps to be** blocked **from accessing Google services, or at minimum blocked from** sensitive and restricted scopes**. Name this `Unverified-App Block – Org-Wide – Pilot` (pilot-scoped first).

> ***Rollback for every setting above:*** *the previous access level / install policy / unconfigured- app setting, recorded before the change. Reverting the specific setting restores prior behavior; tokens already granted are unaffected by policy alone (revoke separately in Stage 5b).*

Pass gate: the config helper's dry-run shows the intended state matches the Stage 2 design with no unexplained breakage before you pass `--apply`.

Stage 4 — Pilot: apply to the pilot OU/group

1. Scope the settings above to `App-Access Pilot – Pilot-OU – v1` (the representative pilot from

Requirement 5). Apply the classification with the helper:

```
python api-access-control-helper.py --plan classification.json --scope pilot-ou --apply
```

1. Have pilot users exercise their normal third-party apps for the agreed soak period.

Pass gate: no legitimate pilot app is broken; any Trusted app still works; blocked/unverified apps are correctly denied (verified in Stage/Validation testing).

Stage 5 — Validate against the gates

1. Run the validation/report tool against the pilot scope:

```
python app-access-validation.py --scope pilot-ou --report validation.json
```

It checks the three gates: **(a)** high-risk (restricted/sensitive) scopes are only held by Trusted apps; **(b)** unverified/unconfigured apps are blocked (or blocked from sensitive scopes); **(c)** the Marketplace install policy is allowlist-only and matches `Trusted Apps – Org-Wide – v1`.

Pass gate: all three gates PASS for the pilot. Any FAIL is remediated (reclassify, adjust setting, or revoke) and re-validated before expansion.

Stage 5b — Revoke risky standing grants (`--apply` gated)

Standing tokens granted before the policy remain until revoked. Use the revocation helper to act on the HIGH-risk grants identified in the inventory, following `Token Revocation – Risky-Grants – Runbook`:

1. Preview which grants would be revoked (safe default — no writes):

```
python risky-grant-revoker.py --inventory app-inventory.json --risk HIGH --dry-run
```

1. After approver sign-off, revoke:

```
python risky-grant-revoker.py --inventory app-inventory.json --risk HIGH --apply
```

It calls the **Directory API** `tokens.delete` for each targeted (user, app) grant and writes an evidence log.

> ***Rollback:*** *revocation forces users to re-consent to legitimately needed apps; there is no "un-revoke." Therefore revoke only vetted-Blocked or clearly abusive grants, communicate to affected users, and keep the evidence log. This is why* `--apply` *is gated and approver-signed.*

Stage 6 — Expand org-wide under change control

1. Within the approved change window (Requirement 6), promote each pilot setting from pilot scope to **org-wide**: the per-app Trusted/Limited/Blocked classification, `Marketplace Allowlist – Org-Wide – v1`, and `Unverified-App Block – Org-Wide – Pilot` → org-wide (rename to `Unverified-App Block – Org-Wide – v1`).

1. Re-run validation org-wide:

```
python app-access-validation.py --scope org --report validation-org.json
```

Pass gate: all three gates PASS org-wide.

1. Operationalize: schedule periodic re-inventory + token review (see Best Practices), and route all new third-party apps through `App-Vetting Workflow – Org-Wide – v1` before they are Trusted.

> ***Org-wide rollback:*** *revert the specific org-wide setting to its recorded prior value; the pilot scope remains governed while you investigate.*

Testing & Validation

Run these procedures after the pilot (Stage 4) and again after org-wide expansion (Stage 6). Each has an explicit **expected pass / fail**. The automated checks are produced by `app-access-validation.py`; the manual checks confirm the Admin console state.

Test 1 — Inventory completeness

- **Procedure:** Run `python oauth-app-inventory.py --days 180 --out app-inventory.json`. Compare the connected-app count to **Admin console** → **Security** → **Access and data control** → **API controls** → **App access control** → **connected apps**.
- **Expected PASS:** counts match within reporting lag; every HIGH-risk and UNKNOWN-scope app is accounted for in the classification.
- **FAIL:** a HIGH-risk app in the console is missing from the inventory, or an UNKNOWN scope is untriaged → investigate auth/scopes and re-run before proceeding.

Test 2 — Gate A: high-risk scopes only held by Trusted apps

• **Procedure:** `python app-access-validation.py --scope <pilot-ou|org> --report validation.json` and

read the **gate_a** result.

• **Expected PASS:** every app holding a **restricted or sensitive** scope appears on `Trusted Apps – Org-Wide – v1`; no Limited/unclassified app holds a sensitive/restricted scope.

• **FAIL:** a non-Trusted app holds a sensitive/restricted scope → reclassify (Trusted after vetting, or Blocked) and, if abusive, revoke via Stage 5b; re-run.

Test 3 — Gate B: unverified/unconfigured apps blocked

• **Procedure:** Read **gate_b** from the validation report; then manually attempt to authorize a brand-new, unlisted test app as a pilot user.

• **Expected PASS:** the report shows the unconfigured-app policy set to block (or block from sensitive/restricted scopes); the manual authorization of a new unlisted app is **denied** (or denied for sensitive scopes) for the pilot user.

• **FAIL:** a new unlisted app obtains sensitive-scope access → confirm `Unverified-App Block – Org-Wide – Pilot` is applied to the pilot scope; correct and re-test.

Test 4 — Gate C: Marketplace allowlist enforced

• **Procedure:** Read **gate_c** from the report; then, as a pilot user, attempt to install a Marketplace app that is **not** on `Marketplace Allowlist – Org-Wide – v1`.

• **Expected PASS:** the install policy is **allowlist-only** and the allowlist matches `Trusted Apps – Org-Wide – v1`; the non-allowlisted install is **blocked**.

• **FAIL:** a non-allowlisted Marketplace app installs → re-check the Marketplace setting scope and the allowlist contents.

Test 5 — Trusted app still works (no false breakage)

• **Procedure:** As a pilot user, use a **Trusted** app that legitimately needs a sensitive scope (e.g. an approved backup/DLP integration).

• **Expected PASS:** the Trusted app authorizes and functions normally — governance did not break a vetted app.

• **FAIL:** a Trusted app is denied → confirm it is on `Trusted Apps – Org-Wide – v1` with the correct

OAuth client ID and that its access level is **Trusted**, not Limited.

Test 6 — Risky-grant revocation (dry-run vs apply)

- **Procedure:** `python risky-grant-revoker.py --inventory app-inventory.json --risk HIGH --dry-run`,

review; then (after sign-off) `--apply`.

- **Expected PASS (dry-run):** the tool lists exactly the (user, app) grants it *would* revoke and

makes **no** change. **PASS (apply):** targeted grants are revoked, an evidence log is written, and a re-inventory no longer shows those grants.

- **FAIL:** dry-run lists grants outside the intended risk set, or apply revokes a Trusted app → stop,

correct the filter/classification, re-run dry-run.

Test 7 — Vetting workflow enforced for a new app

- **Procedure:** Introduce a hypothetical new third-party app request and run it through

`App-Vetting Workflow – Org-Wide – v1`; generate the review packet with the report generator.

- **Expected PASS:** the app is not Trusted/allowlisted until the approver signs the review packet;

the decision is recorded and applied to the correct list.

- **FAIL:** a new app reaches Trusted/allowlist without a recorded decision → the workflow is not being

followed; reinforce the gate.

Overall exit criterion: Gates A, B, and C all **PASS** at the target scope (pilot, then org-wide), no Trusted app is broken (Test 5), risky grants are revoked with evidence (Test 6), and new apps are gated by the vetting workflow (Test 7).

Troubleshooting

Common failure modes and resolutions for OAuth & third-party app-access governance. Artifact names match the Implementation guide.

Auth & API access

`--check-auth` **fails or the tools get 401/403.**

- Cause: ADC not set, or the authenticated identity lacks the read-only Reports/Directory scopes, or

the Admin SDK API is not enabled.

- Fix: run `gcloud auth application-default login` as an admin; enable the **Admin SDK API** in the

automation project; for a service account, authorize **domain-wide delegation** for `admin.reports.audit.readonly` and `admin.directory.user.security` (Requirement 2). Re-run `python oauth-app-inventory.py --check-auth`.

`insufficientPermissions` **on** `tokens.list` **/** `tokens.delete`**.**

- Cause: the delegated identity is not a super-admin equivalent for the Directory `tokens` resource,

or DWD is scoped to the wrong scopes.

• Fix: confirm the account has the security-management privilege and that DWD lists exactly `admin.directory.user.security`. Never add write scopes the tool doesn't need.

`ImportError: googleapiclient`.

• Fix: `pip install google-api-python-client google-auth` (Python 3.10+).

Inventory quality

Inventory count is lower than the Admin console connected-apps count.

• Cause: Reports API/token propagation lag, or a user page failed mid-enumeration (the tool warns per user on stderr).

• Fix: re-run after the reporting window settles; check stderr WARN lines and confirm no large group of users errored. Treat the console count as the source of truth for completeness (Test 1).

Many apps show risk `UNKNOWN`.

• Cause: their scopes aren't in the scope-sensitivity reference (or you're on the built-in starter hints).

• Fix: run `scope-config-helper.py --scope-ref scope-sensitivity.json --inventory app-inventory.json` to list the UNKNOWN scopes, classify them against Google's current OAuth scope documentation (*verify for your tenant*), and re-run the inventory with `--scope-ref`. UNKNOWN is deliberately **not** downgraded to LOW — triage it.

Configuration & policy

A legitimate app broke after applying the classification.

• Cause: an app that legitimately needs a sensitive scope was set to **Limited/Blocked**, or it isn't on `Trusted Apps – Org-Wide – v1` with the correct OAuth client ID.

• Fix: the `api-access-control-helper.py` dry-run flags these as **BREAK-RISK** before apply — heed the flag. To recover: set the app to **Trusted** for the affected scope after vetting, or scope the change back to `App-Access Pilot – Pilot-OU – v1` while you investigate. **Rollback:** revert the per-app access level to its recorded prior value.

Unverified-app block also blocks an app we actually use.

• Cause: the app is unconfigured (not on any list) and `Unverified-App Block – Org-Wide – Pilot` denies it.

• Fix: vet the app via `App-Vetting Workflow – Org-Wide – v1` and add it to the Trusted (or Limited)

list; unconfigured apps are *supposed* to be denied — that is Gate B working. Confirm you piloted before org-wide expansion.

Marketplace Gate C fails: `trusted_not_allowlisted` **/** `only_in_allowlist` **non-empty.**

- Cause: the Marketplace allowlist and `Trusted Apps – Org-Wide – v1` have drifted apart.
- Fix: reconcile — add missing Trusted apps to the allowlist and remove allowlist entries that aren't Trusted, so `Marketplace Allowlist – Org-Wide – v1` == the Trusted list. Re-run validation.

Console path for the unconfigured-app / Marketplace setting isn't where the guide says.

- Cause: Google moves and renames Admin console menus and rolls features out by edition.
- Fix: search the Admin console for "API controls", "app access control", "Marketplace"; confirm the setting under **Security** → **Access and data control** → **API controls**. Where a control's availability is edition/rollout-dependent, verify it exists for your tenant (author's flag, not an error).

Revocation

`risky-grant-revoker.py --apply` **reports** `error:` **for some grants.**

- Cause: the (user, app) grant no longer exists (already revoked / user removed), or a transient API error.
- Fix: errors are recorded per grant in the evidence log and don't stop the run. Re-run dry-run to confirm the remaining set; already-gone grants are safe to ignore.

Users complain an app stopped working after revocation.

- Cause: revocation forces re-consent; this is expected for anything revoked.
- Fix: for a *legitimately needed* app, confirm it's Trusted/allowlisted and have the user re-authorize. This is why revocation is `--apply`-gated, approver-signed, and communicated. There is no "un-revoke" — only re-consent.

Validation exits non-zero in a change-control pipeline.

- Cause: `app-access-validation.py` returns exit code 1 when any gate FAILs (by design).
- Fix: read `validation.json` for the failing gate's offenders, remediate, and re-run until OVERALL = PASS before promoting scope from pilot to org-wide.

Best Practices

Durable practices for keeping OAuth & third-party app access governed after the initial rollout.

Governance & process

- **Vet before you trust.** Every new third-party app goes through `App-Vetting Workflow – Org-Wide – v1 *before* it is added to` Trusted Apps – Org-Wide – v1` or the Marketplace allowlist. "Trusted" is a decision with a named approver and a recorded rationale, never a default.

• **Default-deny the unknown.** Keep `Unverified-App Block – Org-Wide – v1` set to block unconfigured

apps from sensitive/restricted scopes at minimum. New apps should have to earn access, not lose it.

• **Allowlist the Marketplace.** Keep Marketplace on allowlist-only. An open Marketplace is a

continuous stream of un-vetted grants.

• **Least scope, not least app.** Prefer apps that request the narrowest scopes for the job. When two

apps do the same task, favor the one that avoids restricted scopes (full Gmail/Drive/admin).

Ongoing hygiene

• **Re-inventory on a schedule.** Run `oauth-app-inventory.py` monthly (or per your risk cadence).

New HIGH-risk or UNKNOWN-scope apps are the signal to trigger vetting or revocation.

• **Review tokens, not just policy.** Policy governs *new* grants; **standing tokens persist** until

revoked. Periodically run the risky-grant review (dry-run first) and revoke vetted-Blocked or abandoned grants via `Token Revocation – Risky-Grants – Runbook`.

• **Watch scope drift.** Google's restricted/sensitive scope classification changes; keep the

scope-sensitivity reference current and treat any newly-restricted scope as a re-review trigger.

• **Correlate with sign-in and audit signals.** Cross-reference high-risk OAuth grants with the

security investigation / audit logs; an unusual app + unusual sign-in together is a stronger signal than either alone.

Safety & operations

• **Pilot every tightening.** Any change that could block a legitimate app (new unverified-app policy,

Marketplace change, reclassifying an app to Limited/Blocked) goes to `App-Access Pilot – Pilot-OU – v1` first with the documented rollback ready.

• **Communicate revocations.** Revoking a grant forces re-consent; tell affected users why and what to

re-authorize, so a security action doesn't read as an outage.

• **Keep evidence.** Retain the validation reports, revocation evidence logs, and vetting decision

records — they are the audit trail that the surface is governed.

• **Automate safely.** Run the tools read-only/dry-run by default; require `--apply` (and approver

sign-off) for writes and revocations. **Never** commit or export a service-account key — use ADC or workload identity, scoped to the minimum read-only APIs.

Alignment

• **Tie trusted apps to business owners.** Every Trusted app should have a named business owner

accountable for it; orphaned trusted apps are a review trigger.

• **Feed the joiner/mover/leaver process.** When a vendor relationship ends, Block the app and revoke its grants as part of offboarding — don't let a dead vendor keep a live token.

• **Verify edition-dependent controls.** Some app-access and security-center capabilities depend on the Workspace edition and Google's current rollout; confirm availability in your tenant rather than assuming a control exists.

Deliverables Summary

This solution delivers a complete, self-guided OAuth & third-party app-access governance package for Google Workspace: the documentation set plus five authentic Python automation tools.

Documentation

• **Solution Overview** — the OAuth risk model (scopes, connected apps, the trusted/limited/blocked API access control model, Marketplace allowlisting, unverified-app blocking) and the reference design with named artifacts.

• **Requirements Guide** — the solution-specific prerequisites (super-admin/API-controls privilege, Admin SDK + delegated automation identity, scope-sensitivity reference, vetting authority, representative pilot, change window), each with verify/implement steps, building on the shared Google-category global requirements.

• **Implementation Guide** — exact ordered steps: inventory apps + scopes, classify by risk, configure API access control + Marketplace allowlist + unverified-app block, pilot, validate against the gates, revoke risky grants, and expand org-wide — with pass/fail gates and rollback at each stage.

• **Testing & Validation** — seven procedures with expected pass/fail, covering the three governance gates plus no-false-breakage and revocation checks.

• **Troubleshooting** — common failure modes (auth/scope errors, reporting lag, broken legitimate apps, revocation side-effects) and resolutions.

• **Best Practices** — durable app-access hygiene: vet-before-trust, default-deny the unknown, scheduled re-inventory and token review, and safe automation.

• **Executive Summary** — the outcome, approach, and why the change is safe and governed.

Python tools (`scripts/`)

All tools are **read-only or dry-run by default**; writes/revocations require `--apply`. They authenticate via **Application Default Credentials** or a delegated service account — **no hardcoded credentials, no exported keys, no secrets in code**. Python 3.10+; `pip install google-api-python-client google-auth`.

1. `oauth-app-inventory.py` — *Assessment / discovery (read-only).* Enumerates connected

third-party apps, their OAuth scopes, and per-app grant counts from the Admin SDK Reports API (OAuth-token activity) and Directory API (domain tokens); risk-scores each app (HIGH/MEDIUM/LOW/UNKNOWN) using the scope-sensitivity reference. Produces `App Inventory – Org-Wide – v1`.

1. `api-access-control-helper.py` — *Configuration helper (dry-run default).* Takes a classification plan (Trusted/Limited/Blocked per app) and shows current-vs-intended access level, flagging changes that could break a legitimate app. Applies only with `--apply`; supports pilot scoping. Drives `Trusted/Limited/Blocked Apps – Org-Wide – v1`.

1. `app-access-validation.py` — *Validation / report generator (read-only).* Checks the three governance gates — (A) high-risk scopes only held by Trusted apps, (B) unverified apps blocked, (C) Marketplace allowlist-only — and emits a pass/fail report and the approver review packet.

1. `risky-grant-revoker.py` — *Policy/guardrail — token revocation (`--apply` gated).* From the inventory, targets HIGH-risk (or vetted-Blocked) standing grants and revokes them via the Directory API `tokens.delete`, writing an evidence log. Dry-run by default; executes `Token Revocation – Risky-Grants – Runbook`.

1. `scope-config-helper.py` — *Scope/config helper (read-only).* Loads and validates the scope-sensitivity reference (restricted/sensitive/basic tiers), reports scopes seen in the inventory that are UNKNOWN to the reference (for manual triage), and emits a reviewable scope-classification file the other tools consume.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

api-access-control-helper.py

app-access-validation.py

oauth-app-inventory.py

risky-grant-revoker.py

scope-config-helper.py